GoalSync

Your Goals. Your Journey. Together.

Complete setup, deployment and testing guide for beginners

This guide takes you from a downloaded ZIP file to a live, private website that only you and your friend can use. Every step says exactly where to do it and what you should see. You do not need to know how to code.

Part	What you do	Time
1-3	Download the project, install Node.js, VS Code and Git, open the folder	15 min
4-5	Install the libraries, create your Supabase project	15 min
6-7	Create the database and choose the two allowed emails; set up sign-in	10 min
8	Connect the project to Supabase and run it on your computer	10 min
9-11	Put the code on GitHub and publish the website	15 min
12-14	Set the live address in Supabase, create the two accounts, connect them	10 min
15-16	Test everything; learn how to update the site later	20 min

> **Good to know:** Menu names in Supabase and GitHub change from time to time. If a label is slightly different from this guide, look for the closest match. The steps and the values you enter stay the same.

How GoalSync fits together

GoalSync has two halves. Understanding them makes every later step easier.

Half	What it is	Where it lives
The website (frontend)	The pages, buttons and charts you see. These are plain files.	GitHub Pages (free hosting)
The data and logins (backend)	Accounts, passwords, tasks, friend connection and security rules.	Supabase (free database)

Why GitHub Pages alone is not enough

GitHub Pages only hands out static files to anyone who asks. It cannot keep secrets, check passwords or protect a database. That is why GoalSync keeps everything private in **Supabase**, and the website simply talks to it. If you ever need a server of your own (for example to delete accounts from inside the app), use a host that supports server code such as Netlify, Vercel or Cloudflare. For this project, GitHub Pages is a good fit.

How your data stays private

- **Only two emails can sign up.** You will list them in Supabase. Anyone else is rejected by the database itself, not by the website.
- **Row Level Security (RLS)** is switched on for every table. The database checks who is asking on every single request. Hiding the website address or hiding buttons is not what protects you.
- **Friends can read, never write.** After you both accept the connection, each of you can read the other's tasks. Nobody can change, tick or delete the other person's tasks.
- **The website contains only a public key.** Supabase calls it the "anon" or "publishable" key. It is safe to be visible because RLS decides what it can do. The **service_role** or **secret** key must never be used here, never pasted into a file and never uploaded to GitHub.

> **Public repository:** Your GitHub repository must be **Public** for free GitHub Pages. That is fine: the code contains no passwords and no private data. Your tasks live in Supabase and are protected by login and RLS.

Part 1. Download and extract the project

1. Download **GoalSync.zip** from this chat and save it somewhere easy, such as your Documents folder.
2. **Windows:** right-click the ZIP file, choose **Extract All...**, then **Extract**. **Mac:** double-click the ZIP file. A folder named **goalsync** appears.
3. Open the new **goalsync** folder. You should see files such as **package.json**, **README.md**, and folders named **src** and **supabase**. If you only see one more folder inside, open it: the right folder is the one that directly contains **package.json**.

> **Tip:** Keep the folder somewhere simple, with no strange characters in its path, for example **Documents/goalsync**.

Part 2. Install Node.js, VS Code and Git

These three free programs are the tools you need. Install them once.

Node.js (runs the project on your computer)

1. Go to **nodejs.org** and download the version marked **LTS** (version 22 or newer).
2. Run the installer and accept the default options.
3. Restart your computer if the installer asks.

Visual Studio Code (where you will type commands)

1. Go to **code.visualstudio.com**, download the version for your computer, and install it with the default options.

Git (sends your code to GitHub)

1. Go to **git-scm.com**, download Git for your computer and install it with the default options.

You will check that all three installed correctly in Part 3, after you open the terminal.

Part 3. Open the project in VS Code

1. Open **Visual Studio Code**.
2. Click **File > Open Folder...** (on Mac: **File > Open...**), choose the **goalsync** folder (the one that contains **package.json**) and click **Select Folder** / **Open**.
3. If VS Code asks "Do you trust the authors?", click **Yes, I trust the authors**.
4. Open the terminal: click **Terminal > New Terminal** in the top menu. A panel appears at the bottom. It is already inside your project folder.

Every command in this guide is typed into this terminal, then you press **Enter**. Now check your tools:

Where to run: VS Code terminal (bottom panel)

```
node -v
npm -v
git --version
```

Each line should print a version number. **node -v** should show **v22** or higher. If you see "not recognized" or "command not found", close VS Code completely, open it again and retry. If it still fails, reinstall that program.

Part 4. Install the project's libraries

Where to run: VS Code terminal

```
npm install
```

This downloads the libraries GoalSync needs into a folder called **node_modules**. It takes one or two minutes. Warnings in yellow are normal. Red lines that say **ERR!** are not: see Troubleshooting at the end.

Do this once. You do not upload **node_modules** anywhere: the **.gitignore** file already keeps it, and your secret settings, out of GitHub.

Part 5. Create your Supabase project

1. Go to **supabase.com** and click **Start your project**. Sign up (the GitHub sign-in button is the quickest).
2. Create an **organization** if asked (any name, the free plan is enough).
3. Click **New project**. Name it **goalsync**.
4. For **Database password**, click **Generate a password** and save it in a password manager or a safe note. You will not need it for this guide, and you must never paste it into any project file.
5. Pick the **region** closest to you, then click **Create new project**.
6. Wait about two minutes until the dashboard finishes setting up.

> **Tip:** Free Supabase projects can be paused after a long period of no activity. If your site stops working after a long break, open the Supabase dashboard and click **Restore project**.

Part 6. Create the database and the two allowed members

You will run two small scripts. They are inside your project folder, so you can open them in VS Code, copy everything, and paste into Supabase.

6a. Create tables and security rules

1. In VS Code, in the left file list, open **supabase** > **schema.sql**. Press **Ctrl+A** (Mac: **Cmd+A**) then **Ctrl+C** (Mac: **Cmd+C**) to copy everything.
2. In your browser, open your Supabase project. In the left menu click **SQL Editor**, then **New query**.
3. Paste the script (**Ctrl+V**) and click **Run** (bottom right).
4. You should see **Success. No rows returned**. If you see a red error, see Troubleshooting. Do **not** run this script twice: the second run will say things "already exist".

This script creates the member list, profiles, tasks, friend connections and task history, switches on Row Level Security for each, and adds the helper functions. Every section in the file has a comment explaining it in plain words.

6b. Choose the two people who may use GoalSync

1. In VS Code open **supabase** > **add_members.sql**.
2. Replace **your-email@example.com** with your real email and **your-friend@example.com** with your friend's real email. Keep the quotes. These must be the emails you will use to sign up.
3. Copy the whole file, go to Supabase **SQL Editor > New query**, paste, and click **Run**.
4. To check, create another new query, type the line below, and run it. You should see exactly your two emails.

Where to run: Supabase SQL Editor (in your browser)

```
select email from public.allowed_members;
```

> **Important:** Run 6b **before** anyone signs up. Only emails in this list can ever create an account. The database accepts a maximum of two.

6c. Confirm the security switch is on

1. In Supabase click **Table Editor** in the left menu. Open each table (**tasks**, **profiles**, **friendships**, **task_events**, **allowed_members**). None of them should show a warning that RLS is disabled.

Part 7. Set up sign-in (Authentication)

1. In Supabase click **Authentication** in the left menu.
2. Find the sign-in methods page (called **Sign In / Providers**, or **Providers** in older dashboards) and open **Email**. Make sure it is **enabled**.
3. Keep **Confirm email** turned **on**. This makes each person prove they own their email, which is what makes the member list trustworthy.
4. On the same sign-in settings page, make sure **anonymous sign-ins** are turned **off** (they are off by default).
5. Optional: set the minimum password length to **8** (the website already asks for 8).
6. Open **Authentication > URL Configuration**. For now (testing on your own computer) set **Site URL** to **http://localhost:5173**.
7. Under **Redirect URLs** click **Add URL** and add **http://localhost:5173/****. Click **Save**.

You will come back to this page in Part 12 to add your live website address.

Part 8. Connect the project to Supabase and run it

8a. Find the two values you need

1. In Supabase open **Project Settings** (gear icon, bottom left) > **API Keys** (or **API**). You can also click the **Connect** button at the top of the project.
2. Copy the **Project URL**. It looks like **https://abcdxyz.supabase.co**.
3. Copy the **public key**: the **Publishable key** (starts with sb_publishable_) or the legacy **anon** key (a very long text starting with eyJ). Either one works. If you only see a Publishable key section with nothing in it, click **Create new API Keys**.

> **Important: Never** copy the **service_role** or **secret** key. If a key says "secret" or "service", it is the wrong one.

8b. Create your settings file

Where to run: VS Code terminal (Mac or Linux)

```
cp .env.example .env.local
```

Where to run: VS Code terminal (Windows)

```
copy .env.example .env.local
```

1. In VS Code's file list, click the new file **.env.local** (it is in the main project folder, next to package.json).
2. Replace the two placeholder values so the file looks like this (use your own values, no quotes, no spaces):

Where to run: Inside the file .env.local (edit in VS Code, then save with Ctrl+S)

```
VITE_SUPABASE_URL=https://abcdxyz.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxx
```

This file stays on your computer. It is listed in **.gitignore**, so it is not uploaded to GitHub.

8c. Run GoalSync on your computer

Where to run: VS Code terminal

```
npm run dev
```

The terminal prints a line like **Local: http://localhost:5173/**. Hold Ctrl (Mac: Cmd) and click it, or type that address into your browser. You should see the GoalSync login page. If you see "One more setup step", re-check Part 8b, then press **Ctrl+C** in the terminal and run **npm run dev** again.

To stop the app, click inside the terminal and press **Ctrl+C**.

Tip: You can create your account here already (Sign up page) with an email from your member list. The confirmation email link opens localhost, which only works on this computer while the app is running. Alternatively, wait and create both accounts on the live site in Part 13. Both ways work.

Part 9. Create a GitHub repository

1. Go to **github.com** and sign in (create a free account if needed).
2. Click the **+** at the top right, then **New repository**.
3. Repository name: **goalsync** (any name works, but remember it).
4. Choose **Public**.
5. Do **not** tick "Add a README", "Add .gitignore" or any license. Leave the repository empty.
6. Click **Create repository**. Keep the page open: it shows your repository address, like **https://github.com/YOUR-USERNAME/goalsync.git**.

Part 10. Upload the code to GitHub

First, tell Git who you are. Do this only once per computer. Use your real name and the email of your GitHub account.

Where to run: VS Code terminal

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Now send the project. Replace **YOUR-USERNAME** with your GitHub username (and **goalsync** if you chose a different repository name).

Where to run: VS Code terminal

```
git init
git add .
git commit -m "First version of GoalSync"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/goalsync.git
git push -u origin main
```

A window or browser tab may ask you to sign in to GitHub. Approve it. When the push finishes, refresh your repository page on GitHub: you should see all your files.

Important: Before you push, run **git status**. The file **.env.local** must **not** appear in the list. If it does, stop and check that **.gitignore** exists in the project folder.

Tip: Prefer a point-and-click tool? **GitHub Desktop** (desktop.github.com) can publish the folder instead. Make sure the hidden **.github** folder is included, because it holds the automatic deployment file.

Part 11. Publish the website (GitHub Pages)

GoalSync includes an automatic deployment file (**.github/workflows/deploy.yml**). Each time you push code, GitHub builds the site and publishes it. You only need to give it your two public settings and turn on Pages.

11a. Add the two settings as repository secrets

1. On GitHub open your repository and click **Settings** (top tab).
2. In the left menu click **Secrets and variables > Actions**.
3. Click **New repository secret**. Name: **VITE_SUPABASE_URL**. Secret: your Supabase Project URL. Click **Add secret**.
4. Click **New repository secret** again. Name: **VITE_SUPABASE_ANON_KEY**. Secret: your public Publishable/anon key. Click **Add secret**.

The names must be exactly as written, in capital letters. (GitHub calls them "secrets" but these two values are public by design. The truly secret keys are never needed.)

11b. Turn on GitHub Pages

1. Still in **Settings**, click **Pages** in the left menu.
2. Under **Build and deployment**, set **Source** to **GitHub Actions**.

11c. Run the deployment

1. Click the **Actions** tab. The first push may already have run, but without the secrets it will have failed or built an unconfigured site.
2. Click the newest run named **Deploy to GitHub Pages**, then **Re-run all jobs**. (If nothing is listed, click the workflow name on the left and **Run workflow**.)
3. Wait 1 to 3 minutes until both steps (**build** and **deploy**) show a green tick.
4. Your website address is **https://YOUR-USERNAME.github.io/goalsync/**. Open it: you should see the GoalSync login page.

> **Important:** If you named the repository **YOUR-USERNAME.github.io**, the site lives at the root address instead. In that case open **.github/workflows/deploy.yml** and change the line **VITE_BASE_PATH: /${{ github.event.repository.name }}/** to **VITE_BASE_PATH: /**, then push again.

Part 12. Tell Supabase your live website address

Supabase only sends people back to addresses you approve. Without this step, email links (confirm email, reset password) go to the wrong place.

1. In Supabase open **Authentication > URL Configuration**.
2. Set **Site URL** to your live address, including the slash at the end: **https://YOUR-USERNAME.github.io/goalsync/**
3. Under **Redirect URLs**, add **https://YOUR-USERNAME.github.io/goalsync/**** and keep **http://localhost:5173/**** if you want to keep testing locally.
4. Click **Save**.

> **Tip:** Capital letters matter. Use your GitHub username exactly as it appears in your live address.

Part 13. Create the two member accounts

1. Open your live website. Click **Create your account**.
2. **Member 1** (you): enter a display name, the email you put in the member list, a password of 8 or more characters, and click **Create account**.
3. Open your email inbox, find the message from Supabase, and click the confirmation link. (Check spam if it does not arrive in a few minutes.)
4. You are taken to GoalSync and signed in. Your name appears at the bottom of the left menu (on a phone: Settings).
5. **Member 2** (your friend) does the same on their own device with the other email from your list.

If someone uses an email that is not on the list, they see "This email is not on the member list" and no account is created.

> **Tip:** Do not share your password. Each person uses their own login. If someone forgets a password, use **Forgot your password?** on the login page.

Part 14. Connect the two accounts

1. **Member 1:** open **Settings**. Under **Friend connection**, type Member 2's account email and click **Send request**.
2. GoalSync always replies with the same message, whether or not that email belongs to a member. This stops strangers from guessing who has an account.
3. **Member 2:** log in and open **Settings**. The request from Member 1 appears. Click **Accept**.
4. Both: Settings now says **Connected with ...**. The dashboard shows your friend's progress card and **Friend's View** in the menu shows their details.

What is shared: display name, task titles, categories, dates, times, completed or pending status and completion time, and the progress calculated from them. **What is never shared:** email, password, appearance settings and the status-change log. Your friend can only read. Press **Disconnect** any time: sharing stops immediately and nobody's tasks are deleted.

Part 15. Test the complete application

Open **docs/TESTING_CHECKLIST.md** in VS Code (or in your repository on GitHub) and tick every line using two browsers, one per member. The checklist covers: private access, sign-up blocking, adding/editing/completing/deleting tasks, 30-day repeating tasks, past history, charts, the read-only friend view, colour themes, light/dark/system mode, and phones.

The most important checks

Check	What you should see
Open the site while signed out	Only the login page. Typing /history sends you to login.
Sign up with an email not on the list	A message saying it is not on the member list.
Tick a task, refresh the page	It stays ticked; progress and streak updated.
Friend's View	Your friend's tasks, with no checkboxes, edit or delete buttons.
Each person changes colour/mode	Each person keeps their own choice after logging out and in.
Phone	Bottom menu, round "+" button, no sideways scrolling.

How progress is calculated

- **Completion %** = completed tasks / total tasks in the range x 100, rounded. With no tasks you see a dash, not 0%.
- Tasks dated **after today are not counted** ("upcoming"), so scheduling ahead never lowers your score.
- **Perfect day:** at least one task and all done. **Streak:** perfect days in a row ending today. If today is not finished, the streak continues from yesterday. A day with no tasks breaks it.
- The date range used is written on screen above every figure and chart.

Part 16. Update the website after changing the code

1. Edit files in VS Code and save them (**Ctrl+S**).
2. Check your change on your computer first: run **npm run dev** and open http://localhost:5173.
3. Run the automated tests (optional but recommended):

Where to run: VS Code terminal

```
npm test
```

1. Send the change to GitHub. GitHub rebuilds and republishes automatically (about two minutes):

Where to run: VS Code terminal

```
git add .
git commit -m "Describe your change here"
git push
```

1. Watch the **Actions** tab on GitHub for the green tick, then refresh your live site (press **Ctrl+Shift+R** / **Cmd+Shift+R** to bypass the cache).

Important: Changes to the **database** (new columns, new rules) are different: they are made in Supabase SQL Editor, not by pushing code. Never turn off Row Level Security to "make something work".

Appendix A. Project map

Where	What it does
supabase/schema.sql	Tables, indexes, constraints, triggers, Row Level Security policies, friend functions. Run once.
supabase/add_members.sql	The two allowed emails. Edit, then run.
supabase/tests/	Security tests for developers. Never run them on your real project.
src/lib/dates.ts	All date handling (calendar days, no time-zone shifts).
src/lib/stats.ts	All percentages, streaks, ranges and chart data, in one place.
src/lib/themes.ts	The six colour themes.
src/context/	Login state, saved appearance, task data and friend data, notifications.
src/pages/	Login, Sign up, Dashboard, Past History, Analytics, Friend's View, Settings.
src/components/	Reusable pieces: ProgressRing, TaskList, Calendar, forms, dialogs.
src/index.css	All styling, including light/dark mode and mobile layout.
.github/workflows/deploy.yml	Automatic publishing to GitHub Pages.
.env.example	Template for your private settings file **.env.local**.

Appendix B. What was tested, and what you still need to check

Honest status of this project when it was delivered.

Tested by automated checks

- The project builds and passes the TypeScript check.
- **20 logic tests**: dates without time-zone shifts, completion percentage, streaks, date ranges, chart series, category statistics.
- **9 interface tests** that run the real app against a fake in-memory database: dashboard numbers, ticking and unticking, add-task validation, 30-day repeat with no duplicates, delete confirmation, past history, the read-only friend view, all analytics sections, saving colour and mode, the not-connected state.
- **45 database security checks** on a real PostgreSQL 16 with a stand-in for Supabase's login tables: blocked sign-ups, no signed-out access, members cannot read or change each other's rows, friends can only read, owner cannot be changed, completion time set by the database, status history, disconnect stops access.

Not tested yet (you will do these)

- A real Supabase project: real emails, real sessions and the real login system.
- The live GitHub Pages deployment and the live web address.
- Real browsers and real phones. The layout is written to be responsive but nobody has looked at it on a device yet. Use the checklist and tell me what looks wrong.

Known limits

- Exactly two members, enforced by the database.
- Deleting an account is not inside the app. In Supabase open **Authentication > Users** and delete the user; their tasks are deleted with them.
- Repeat-daily creates 30 linked tasks, not an endless rule. Editing one does not change the rest. Deleting offers "this task" or "this and later repeats".
- Supabase's built-in email sender has a small hourly limit. Wait a few minutes between attempts.

Appendix C. Troubleshooting

Problem	What to do
"npm is not recognized" / "command not found"	Node.js is missing. Install the LTS version from nodejs.org, close VS Code fully, reopen it, retry.
Page says "One more setup step"	The file must be named exactly **.env.local**, sit next to package.json, and contain both values. Stop the app (Ctrl+C) and run **npm run dev** again.
"This email is not on the member list"	Run supabase/add_members.sql with the right emails, or fix a typo. Check with: select email from public.allowed_members;
"Database error saving new user"	The member list blocked it, or schema.sql did not finish. Check the member list first.
schema.sql says "already exists"	It already ran. Do not run it twice.
No confirmation or reset email	Check spam. Wait a few minutes (hourly limit). Confirm Authentication > URL Configuration is set.
Email link opens localhost or an error page	Set Site URL and Redirect URLs to your live address (Part 12).
Live site is blank or 404	Settings > Pages > Source must be **GitHub Actions**. Check the Actions tab for a red cross and read the message.
Build fails or site shows setup page	The two repository secrets are missing or misspelled. Names: VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY. Then re-run the job.
"You do not have permission to do that"	Row Level Security blocked it. Usually the account email is not on the member list, or schema.sql was only partly run.
Friend's tasks do not show	Settings must say "Connected". A pending request shares nothing. Switch tabs or refresh.
"Invalid API key"	Wrong value or an extra space. Use the Project URL and the Publishable/anon key, never the secret key.
Site stopped working after weeks	Open Supabase and restore the paused project.

Still stuck? Press **F12** in your browser, open the **Console** tab, and copy the red message. Send me that exact text with the step you were on.

Appendix D. Command cheat sheet

Command	Where	What it does
npm install	VS Code terminal	Downloads libraries (once, and after pulling new code)
npm run dev	VS Code terminal	Runs GoalSync at http://localhost:5173
npm test	VS Code terminal	Runs the automated tests
npm run build	VS Code terminal	Builds the website into the dist folder (GitHub does this for you)
git add .	VS Code terminal	Selects all changed files
git commit -m "message"	VS Code terminal	Saves a snapshot with a note
git push	VS Code terminal	Sends the snapshot to GitHub, which republishes the site
Ctrl+C	Inside the terminal	Stops the running app